

Workshop 3

Composing with Algorithms
<http://www.bjarni-gunnarsson.net>

```

(
  NF(\iop, {|freq=78, mul=1.0, add=0.0|
    var noise = LFNoise1.ar(0.001).range(freq, freq + (freq * 0.1));
    var osc = SinOsc.ar([noise, noise * 1.04, noise * 1.02, noise * 1.08],0,0.2);
    var out = DFm1.ar(osc,freq*4,SinOsc.kr(0.01).range(0.92,1.05),1,0,0.005,0.7);
    HPF.ar(out, 40)
  }).play;
)

(
  NF(\dsc, {|freq = 1080|
    HPF.ar(
      BBandStop.ar(Saw.ar(LFNoise1.ar([19,12]).range(freq,freq*2), 0.2).excess(
        SinOsc.ar( [freq + 6, freq + 4, freq + 2, freq + 8])),
        LFNoise1.ar([12,14,10]).range(100,900),
        SinOsc.ar(20).range(9,11)
      ), 80)
    ).play;
)

var <>pindex, <>cindex;

initialize {
  if(pindex.isNil, { pindex = 1000 });
  if(cindex.isNil, { cindex = 2000 });
}

clearProcessSlots {
  pindex = 1000;
  (this.pindex - 1000).do{|i| this[this.pindex+i] = nil; }
}

clearOrInit {|clear=true|
  if(clear == true, { this.clearProcessSlots() }, { this.initialize() });
}

transform {|process, index|
  if(index.isNil && pindex.isNil, {
    this.initialize();
  });

  pindex = pindex + 1;
  this[pindex] = \filter -> process;
}

control {|process, index|
  var i = index;

  if(i.isNil, {
    this.initialize();
    cindex = cindex + 1;
    i = cindex;
  });

  this[i] = \pset -> process;
}

(
  NF(\depfm, {|freqMin=5, freqMax=20, mul=20, add=80, rate=0.5, modFreq=2100, index=0.3, amp=0.2|
    var trig, seq, freq;
    trig = Dust.kr(rate);
    seq = Diwhite(freqMin, freqMax, inf).midicps;
    freq = Demand.kr(trig, 0, seq);
    HPF.ar(PMOsc.ar(LFCub.kr([freq, freq/2, freq/3, freq/4], 0, mul, add),
      LFNoise1.ar(0.3).range(modFreq,modFreq*2), index) * amp, 50)
  }).play;
)

```

Tasks

Exercises

See: Exercises.scd

```

(
  NF(\iop, {|freq=78, mul=1.0, add=0.0|
    var noise = LFNoise1.ar(0.001).range(freq, freq + (freq * 0.1));
    var osc = SinOsc.ar([noise, noise * 1.04, noise * 1.02, noise * 1.08],0,0.2);
    var out = DFm1.ar(osc,freq*4,SinOsc.kr(0.01).range(0.92,1.05),1,0,0.005,0.7);
    HPF.ar(out, 40)
  }).play;
)

(
  NF(\dsc, {|freq = 1080|
    HPF.ar(
      BBandStop.ar(Saw.ar(LFNoise1.ar([19,12]).range(freq,freq*2), 0.2).excess(
        SinOsc.ar( [freq + 6, freq + 4, freq + 2, freq + 8])),
        LFNoise1.ar([12,14,10]).range(100,900),
        SinOsc.ar(20).range(9,11)
      ), 80)
    ).play;
)

var <>pindex, <>cindex;

initialize {
  if(pindex.isNil, { pindex = 1000 });
  if(cindex.isNil, { cindex = 2000 });
}

clearProcessSlots {
  pindex = 1000;
  (this.pindex - 1000).do{|i| this[this.pindex+i] = nil; }
}

clearOrInit {|clear=true|
  if(clear == true, { this.clearProcessSlots() }, { this.initialize() });
}

transform {|process, index|
  if(index.isNil && pindex.isNil, {
    this.initialize();
  });

  pindex = pindex + 1;
  this[pindex] = \filter -> process;
}

control {|process, index|
  var i = index;

  if(i.isNil, {
    this.initialize();
    cindex = cindex + 1;
    i = cindex;
  });

  this[i] = \pset -> process;
}

(
  NF(\depfm, {|freqMin=5, freqMax=20, mul=20, add=80, rate=0.5, modFreq=2100, index=0.3, amp=0.2|
    var trig, seq, freq;
    trig = Dust.kr(rate);
    seq = Diwhite(freqMin, freqMax, inf).midicps;
    freq = Demand.kr(trig, 0, seq);
    HPF.ar(PMOsc.ar(LFCub.kr([freq, freq/2, freq/3, freq/4], 0, mul, add),
      LFNoise1.ar(0.3).range(modFreq,modFreq*2), index) * amp, 50)
  }).play;
)

```

Older

Exercises

1. Develop a method for selecting between various pre-configured Pbinds so that they can be played in different order in time.
2. Create a sequencing process that plays at least two different sections. Each of the section should contain parameters that control its behaviour and the order of sequences should be flexible.
3. Create three different snapshots of synthesiser configuration using the SynthDef designer tool. Later, implement at least two different patterns playing these synthdefs.
4. Implement at least one transition from a certain material to another set

Exercises

1. Employ a Tdef to create a short sequence of at least two different **Ndefs** that can be changed while the **Tdef** runs.
2. Create a source and effect chain where the source is processed by the effect using **Ndefs**.
3. Create a modulated synth that contains two modulation sources that can be set using **proxies** (LFOs, Envelopes etc.)
4. Create a small pool of live coding snippets that could be triggered when creating a piece of performance. The pool should contain different sources for event, modulation and sound generation.